IAI600 – Introduction to Applied Artificial Intelligence

Lab IV: Support Vector Machines (SVM)

Main topic: Support Vector Machines for Classification and Regression
Datasets: Iris Dataset and California Housing Dataset
Software: Python, Jupyter Notebook, NumPy, Matplotlib, Scikit-learn
Disclaimer: This lab material has been prepared with the assistance of Generative AI and is based on Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow; please report any errors, inaccuracies, or limitations you identify.

Objective of the laboratory: The objective of this lab is to understand and apply Support Vector Machines (SVMs) for classification and regression. Students will investigate linear and nonlinear decision boundaries, understand the role of support vectors and margins, experiment with the C and γ hyperparameters, apply kernel methods, and use SVMs for regression.
The laboratory emphasizes both the practical use of SVM models and the interpretation of how their main hyperparameters affect model behaviour.

Part I – SVMs: Load and Explore the Iris Dataset

SVMs are supervised machine learning algorithms that can be used for both classification and regression. For classification, an SVM attempts to find a decision boundary that separates different classes while maximizing the margin between the classes. The observations closest to the decision boundary are called **support vectors**. A simplified representation is:

Class A | Decision Boundary | Class B

The objective is not simply to find any separating boundary, but to find a boundary with a large margin.

Load and Explore the Iris Dataset

We will first use the **Iris dataset** to investigate SVM classification. The Iris dataset contains measurements of iris flowers belonging to three different species.
The available features include: sepal length, sepal width, petal length, and petal width.
Load the dataset:

```
from sklearn.datasets import load_iris

iris = load_iris()

X = iris["data"]
y = iris["target"]

print(X.shape)
print(y.shape)
print(iris["target_names"])
```

Inspect the feature names:

```
print(iris["feature_names"])
```

For the first SVM experiment, use petal length and petal width.

```
X = iris[
    ["petal length (cm)",
     "petal width (cm)"]
]
```

Create a binary classification problem by selecting Iris-Virginica and non-Virginica examples:

```
y = (iris["target"] == 2)
```

Visualize the data:

```
plt.figure(figsize=(8, 6))

plt.scatter(
    X[y == False, 0],
    X[y == False, 1],
    label="Not Virginica"
)

plt.scatter(
    X[y == True, 0],
    X[y == True, 1],
    label="Virginica"
)

plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.legend()
plt.grid()
plt.show()
```

Task 1.1 – Explore the Classification Problem

Answer:

1. What is a decision boundary?
2. What is a margin?
3. What is a support vector?
4. How many observations are available in the iris dataset?
5. What are the two classes in this experiment?
6. Which two features are being used?
7. Does the scatter plot suggest that the classes can be separated using a linear decision boundary?

Part II – Feature Scaling

SVMs are sensitive to the scale of the input features. If one feature has values that are much larger than another feature, it can strongly influence the geometry of the decision boundary. Use a standard scaler:

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Task 2.1 – Investigate Feature Scaling

Compare the feature ranges before and after scaling.
Answer:

1. Why can feature scaling be important for SVMs?
2. What happens to the mean and scale of the features after standardization?

Part III – Linear SVM Classification

We can train a linear SVM classifier using Scikit-learn.
Use `SVC` with a linear kernel:

```python
from sklearn.svm import SVC

svm_clf = SVC(
    kernel="linear",
    C=1
)

svm_clf.fit(
    X_scaled,
    y
)
```

Generate predictions:

```python
y_pred = svm_clf.predict(
    X_scaled
)

print(y_pred[:10])
```

Calculate the training accuracy:

```python
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(
    y,
    y_pred
)

print("Accuracy:", accuracy)
```

Task 3.1 – Evaluate the Linear SVM

Answer:

1. What accuracy did you obtain?

2. What does the model predict?
3. Is evaluating only the training data sufficient?
4. How could the model be evaluated more reliably?

Relate your answer to the cross-validation concepts from Laboratory II.

Part IV – Visualize the Decision Boundary

For a two-dimensional problem, it is possible to visualize the decision boundary. Create a mesh grid over the feature space:

```
import numpy as np

x0 = np.linspace(
    X_scaled[:, 0].min() - 1,
    X_scaled[:, 0].max() + 1,
    200
)

x1 = np.linspace(
    X_scaled[:, 1].min() - 1,
    X_scaled[:, 1].max() + 1,
    200
)

x0, x1 = np.meshgrid(x0, x1)

X_new = np.c_[
    x0.ravel(),
    x1.ravel()
]

y_pred_grid = svm_clf.predict(
    X_new
)

y_pred_grid = y_pred_grid.reshape(
    x0.shape
)
```

Plot the decision regions together with the observations.

```
plt.figure(figsize=(8, 6))

plt.contourf(
    x0,
    x1,
    y_pred_grid,
    alpha=0.2
)

plt.scatter(
    X_scaled[y == False, 0],
    X_scaled[y == False, 1],
    label="Not Virginica"
)

```

```
plt.scatter(
    X_scaled[y == True, 0],
    X_scaled[y == True, 1],
    label="Virginica"
)

plt.xlabel("Scaled petal length")
plt.ylabel("Scaled petal width")
plt.legend()
plt.grid()
plt.show()
```

Task 4.1 – Interpret the Decision Boundary

Inspect the visualization and explain:

1. Where is the decision boundary?
2. Which observations are closest to the boundary?
3. What happens to the classification of observations near the boundary?

Part V – Support Vectors and the Margin

The observations that determine the position of the SVM decision boundary are called **support vectors**.
Scikit-learn provides access to them through:

```
print(svm_clf.support_)
```

The corresponding observations can be accessed using:

```
print(
    X_scaled[svm_clf.support_]
)
```

Task 5.1 – Investigate Support Vectors

Answer:

1. How many support vectors are there?
2. Where are they located relative to the decision boundary?
3. Why are these observations important to the trained SVM?
4. what might happen if observations close to the decision boundary were removed from the training set.

Part VI – The C Hyperparameter

The C hyperparameter controls the trade-off between having a wide margin and allowing classification errors.
A smaller value of C allows the model to tolerate more training errors and generally results in a wider margin.
A larger value of C places greater importance on correctly classifying the training examples.
Train models using different values of C:

```
C_values = [0.01, 0.1, 1, 10, 100]

for C_value in C_values:

    svm_clf = SVC(
        kernel="linear",
        C=C_value
    )

    svm_clf.fit(
        X_scaled,
        y
    )

    predictions = svm_clf.predict(
        X_scaled
    )

    accuracy = accuracy_score(
        y,
        predictions
    )

    print(
        "C =", C_value,
        "Accuracy =", accuracy
    )
```

Task 6.1 – Investigate C

Compare the models obtained using different values of C.
Answer:

1. What happens when C is very small?
2. What happens when C becomes large?
3. How does C affect the margin?
4. Does a higher training accuracy necessarily indicate better generalization?

Use cross-validation if you want to compare the models more reliably.

Part VII – Nonlinear Classification with Polynomial Features

Not all classification problems can be separated using a straight line. One approach is to transform the original features into additional nonlinear features.
For example, polynomial features can be created using Scikit-learn.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline

poly_svm_clf = Pipeline([
    (
        "poly_features",
        PolynomialFeatures(degree=3)
```

```
    ),
    (
        "scaler",
        StandardScaler()
    ),
    (
        "svm",
        SVC(
            kernel="linear",
            C=10
        )
    )
])

poly_svm_clf.fit(
    X,
    y
)
```

Task 7.1 – Investigate Nonlinear Features

Compare the linear model with the polynomial-feature model.
Discuss:

1. What happens when the polynomial degree becomes larger?
2. What are the possible disadvantages of using very high-degree polynomial features?

Part VIII – Polynomial Kernel

Instead of explicitly creating polynomial features, SVMs can use a **polynomial kernel**.
Create an SVM using a polynomial kernel:

```
poly_kernel_svm = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "svm",
        SVC(
            kernel="poly",
            degree=3,
            coef0=1,
            C=5
        )
    )
])

poly_kernel_svm.fit(
    X,
    y
)
```

Generate predictions and evaluate the model.

```
predictions = poly_kernel_svm.predict(
    X
)

accuracy_score(
    y,
    predictions
)
```

Task 8.1 – Understand the Polynomial Kernel

Explain:

1. What is the purpose of the polynomial kernel?
2. What does the `degree` parameter control?
3. What could happen if the degree is too high?
4. Why can the kernel approach be more convenient than explicitly creating many polynomial features?

RBF Kernel and γ

Another commonly used kernel is the Radial Basis Function (RBF) kernel.
Create an SVM using the RBF kernel:

```
rbf_svm = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "svm",
        SVC(
            kernel="rbf",
            C=1,
            gamma=1
        )
    )
])

rbf_svm.fit(
    X,
    y
)
```

Generate predictions and calculate the accuracy.

```
predictions = rbf_svm.predict(
    X
)

print(
    accuracy_score(
        y,
        predictions
    )
```

```
)
```

Task 8.2 – Investigate γ

Train models using several values of γ.
For example:

```
gamma_values = [
    0.01,
    0.1,
    1,
    10,
    100
]

for gamma_value in gamma_values:

    model = Pipeline([
        (
            "scaler",
            StandardScaler()
        ),
        (
            "svm",
            SVC(
                kernel="rbf",
                C=1,
                gamma=gamma_value
            )
        )
    ])

    model.fit(X, y)

    predictions = model.predict(X)

    print(
        "gamma =", gamma_value,
        "accuracy =",
        accuracy_score(y, predictions)
    )
```

Answer:

1. What happens when γ is very small?
2. What happens when γ becomes very large?
3. How can a very large γ lead to a complex decision boundary?
4. How might an inappropriate value of γ lead to overfitting?

Part IX – Comparing SVM Classification Models

At this point, several SVM models have been created:

- linear SVM,

- linear SVM with different C values,
- polynomial-feature SVM,
- polynomial-kernel SVM,
- RBF-kernel SVM.

Evaluate selected models using cross-validation.
For example:

```python
from sklearn.model_selection import cross_val_score

models = {
    "Linear SVM": svm_clf,
    "Polynomial Kernel": poly_kernel_svm,
    "RBF Kernel": rbf_svm
}

for name, model in models.items():

    scores = cross_val_score(
        model,
        X,
        y,
        cv=5,
        scoring="accuracy"
    )

    print(
        name,
        "Mean accuracy:",
        scores.mean(),
        "Std:",
        scores.std()
    )
```

Task 9.1 – Compare the Models

Create a table containing:

Model	Mean Accuracy	Std. Dev.
Linear SVM		
Polynomial Kernel		
RBF Kernel		

Discuss:

1. How do the models compare?
2. Which hyperparameters have the largest effect?
3. Why should cross-validation be used rather than comparing only training accuracy?

Part X – SVM for Regression

SVMs can also be used for regression. In classification, the objective is to find a suitable decision boundary.
In regression, the objective is to predict a numerical value while controlling the size of the errors.
We will use the California Housing dataset from Labs I and II.
Load the dataset:

```
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(
    as_frame=True
)

X = housing.data
y = housing.target
```

Split the data:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
)
```

Create a pipeline containing scaling and a linear SVR model:

```
from sklearn.svm import LinearSVR

linear_svr = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "svr",
        LinearSVR(
            epsilon=0.5,
            random_state=42
        )
    )
])

linear_svr.fit(
    X_train,
    y_train
)
```

Generate predictions:

```
y_pred = linear_svr.predict(
    X_test
)
```

Calculate the RMSE:

```python
from sklearn.metrics import mean_squared_error

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)

print("RMSE:", rmse)
```

Task 10.1 – Interpret SVM Regression

Answer:

1. How is SVM regression different from SVM classification?
2. What does the ϵ parameter represent?

Part XI – Nonlinear SVM Regression

Scikit-learn also provides the more flexible `SVR` implementation.
Try an RBF-kernel SVR:

```python
from sklearn.svm import SVR

rbf_svr = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "svr",
        SVR(
            kernel="rbf",
            C=100,
            gamma="scale",
            epsilon=0.1
        )
    )
])

rbf_svr.fit(
    X_train,
    y_train
)
```

Generate predictions and calculate RMSE:

```python
y_pred = rbf_svr.predict(
    X_test
)

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
```

```
    )
)

print("RBF SVR RMSE:", rmse)
```

Task 11.1 – Experiment with SVR Hyperparameters

Experiment with different values of: C, γ, ϵ.
Create a small comparison table.

C	γ	ϵ	RMSE

Discuss:

1. Which parameters had the greatest effect?
2. Did increasing C always improve the model?
3. What happened when γ was increased?

Part XII – Classification or Regression?

SVM can be used for both classification and regression, but the objective is different.

	Classification	**Regression**
Goal	Predict a class	Predict a number
Example	Iris species	House price
Model	SVC	SVR
Output	Class label	Numerical value